JeremyDEZETREE_TP5

November 29, 2025

1 TP 5 - Compression d'images par K-means et centres mobiles

Dans cet exercice, on cherche à réduire le nombre de couleur d'une image pour la compresser.

1.0.1 Données

Les données relatives à cet exercice sont à télécharger sur le cours Eureka sous forme d'un fichier compacté zip `Données de l'exercice sur le clustering`.

Il contient notamment un fichier image tiff de 538×538 pixels nommé `bird_large.tiff` :

Cette image est au format 24-bits avec des couleurs codées en RVB (RGB en angais), correspondant à la combinaison de l'intensitée de 3 couleurs de base : rouge, vert et bleu. L'intensité pour chacune de ces couleurs varie entre 0 et 255, nécessitant 1 octet (i.e. 8-bits), soit 16 millions de couleurs possibles.

L'objectif est de réduire ce nombre de couleurs utilisées dans l'image à seulement 16 couleurs et donc de compresser l'image.

On va utiliser pour cela un algorithme K-means. On va calculer 16 couleurs correspondant au centre de centroïds. Et on va remplacer la couleur de chaque pixel par la couleur du centroïd qui en est la plus proche.

Pour réduire le temps de calcul du à la manipulation de l'image au format 538x538 pixels, on va déterminer ces clusters de couleurs en utilisant une image plus petite : `bird_small.tiff` au format 128×128 pixels. Ensuite, on appliquera cette définition des couleurs sur la grande image.

On peut charger l'image par le code :

```
import matplotlib.image as mpimg
import numpy as np
img = mpimg.imread("bird_small.tiff")
```

`img` est alors une matrice à 3 dimensions contenant la position du pixel pour les deux premiers indices, et la valeur de la composante couleur indiquée par le troisième indice. Par exemple, `img[50, 33, 2]` donnera l'intensité de la troisième composante de couleur (bleu) du pixel à la position (50,33).

On peut modifier la couleur du pixel de coordonnées (100,120) de la manière suivante :

```
img[100, 120] = (56, 120, 355)
```

La couleur de ce pixel sera codée par (rouge=56, vert=120, bleu=355).

Le but est d'identifier 16 centroids de clusters à partir de l'image. A chaque centroïd j, une couleur est associé et correspond à un vecteur à 3 composantes selon le codage RVB :
$\mu_j = [r_j, v_j, b_j]$.

Ci-dessous, on désigne par $x_i = [r_i, v_i, b_i]$, le vecteur couleur à 3 composantes RVB pour chaque pixel i de l'image. Ainsi x est stockée sous la forme d'une matrice à 3 colonnes (une pour chaque composante) et à 128×128 lignes si l'on prend la petite image.

De même, μ peut être vue comme une matrice de 16 lignes et 3 colonnes.

NB : on peut cependant continuer de fonctionner avec les coordonnées des pixels dans la matrice (fonctionner avec un indice de ligne, un indice de colonne), il faudra adapter l'algorithme ci-dessous.

1.0.2 Algorithme K-means

1. Commencer par choisir aléatoirement 16 couleurs dans l'image pour caractériser les centroids initiaux, les k-means : $\mu_1, \mu_2, \dots, \mu_k$.
2. Pour chacun des pixels i dans l'image, trouver la valeur de couleur la plus proche parmis les μ_j et enregister l'indice du centroïd correspondant :

$$c_i = \arg\min_j \parallel x_i - \mu_j \parallel^2$$

3. Mettre à jour les couleurs μ_j associées à chacun des 16 centroids connaissant les pixels constituant les clusters (on recalcule la moyenne des couleurs des pixels qui appartiennent au même centroïde avec une formule de barycentre equipondéré) :

$$\mu_j(j) = \frac{\sum_i 1\{c_i = j\} x_i}{\sum_i 1\{c_i = j\}}$$

   où $1\{\dots\}$ désigne la fonction indicatrice qui vaut 1 lorsque l'expression booléenne dans l'accolade est vraie et 0 sinon.

4. Répéter les étapes 2 et 3 jusqu'à la convergence (entre 30 et 100 itérations). Deux possibilités : soit fixer le nombre d'itérations au départ de la boucle, soit entrer dans une boucle de laquelle on sort dès que les couleurs associées aux centroids sont stabilisées.

NB 1 : commencer par un nombre fixe d'itérations avant de mettre au point la seconde alternative.

NB 2 : à l'étape 3, il est possible qu'un centroid ne possède aucun pixel, on risque alors une division par 0 ce qu'il faut détecter pour éviter cette situation.

1.0.3 Application à la grande image

Après la convergence de l'étape précédente, prendre la grande image et remplacer la couleur de chaque pixel par la couleur du centroïd la plus proche et calculée sur la petite image. Puis afficher l'image obtenue avec un code du type :

```
import matplotlib.pyplot as plt
plt.imshow(img)
plt.show()
```

[1]:
```python
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import numpy as np

# 1. Charger la petite image pour l'apprentissage
img_small = mpimg.imread('bird_small.tiff')

# Vérification des dimensions
print(f"Dimensions de l'image small : {img_small.shape}")

# On aplatit l'image pour avoir une matrice (Nombre de pixels x 3 couleurs)
# On divise par 255 si les données ne sont pas déjà entre 0 et 1 (dépend de la␣
↪version de mpimg)
X = img_small.reshape(-1, 3)
if X.max() > 1.0:
    X = X / 255.0

print(f"Forme de la matrice de données X : {X.shape}")
```

```
Dimensions de l'image small : (128, 128, 3)
Forme de la matrice de données X : (16384, 3)
```

[2]:
```python
def init_centroids(X, K):
    """Initialise aléatoirement K centroids parmi les points existants"""
    indices = np.random.choice(X.shape[0], K, replace=False)
    return X[indices]

def find_closest_centroids(X, centroids):
    """
    Pour chaque pixel dans X, trouve l'index du centroid le plus proche.
    Retourne un vecteur d'indices.
    """
    # Calcul de distance vectorisé pour éviter les boucles lentes
    # On utilise le broadcasting de numpy : (N, 1, 3) - (K, 3)
    distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
    return np.argmin(distances, axis=1)

def compute_centroids(X, idx, K):
    """
    Met à jour la position des centroids en calculant la moyenne des points␣
↪assignés.
    """
    centroids = np.zeros((K, 3))
    for k in range(K):
        points = X[idx == k] # Tous les pixels assignés au cluster k
        if len(points) > 0:
            centroids[k] = np.mean(points, axis=0)
```

```
        # Si aucun point n'est assigné (rare), le centroid reste à 0 ou on␣
↪pourrait garder l'ancien
    return centroids
```

[3]:
```
# Paramètres
K = 16
max_iters = 20 # Tu peux monter à 50 ou 100 si besoin

# Initialisation
initial_centroids = init_centroids(X, K)
centroids = initial_centroids

print("Démarrage de K-Means...")

for i in range(max_iters):
    # 1. Assignation des clusters
    idx = find_closest_centroids(X, centroids)

    # 2. Mise à jour des centroids
    previous_centroids = centroids
    centroids = compute_centroids(X, idx, K)

    # Vérification de la convergence (si ça ne bouge plus, on arrête)
    if np.allclose(centroids, previous_centroids):
        print(f"Convergence atteinte à l'itération {i}")
        break

    if i % 5 == 0:
        print(f"Itération {i}/{max_iters}...")

print("K-Means terminé.")
# centroids contient maintenant nos 16 couleurs optimales
```

```
Démarrage de K-Means…
Itération 0/20…
Itération 5/20…
Itération 10/20…
Itération 15/20…
K-Means terminé.
```

[4]:
```
# 1. Charger la grande image
img_large = mpimg.imread('bird_large.tiff')
original_shape = img_large.shape

# 2. Préparer les données (aplatir)
X_large = img_large.reshape(-1, 3)
# Normalisation si nécessaire (comme pour la petite image)
```

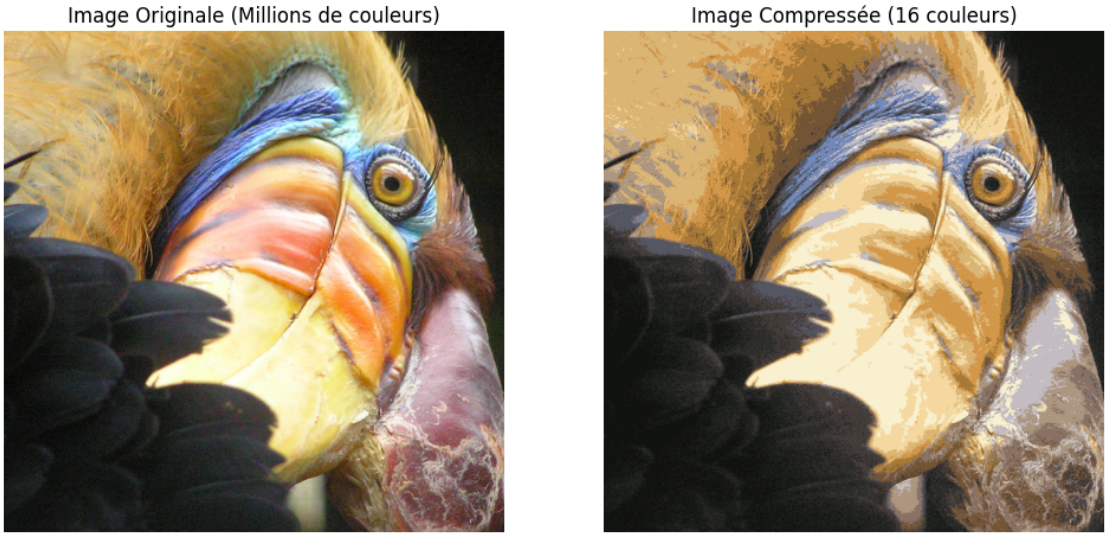
```python
if X_large.max() > 1.0:
    X_large = X_large / 255.0

# 3. Trouver le cluster le plus proche pour chaque pixel de la grande image
# On utilise les centroids calculés à l'étape précédente
idx_large = find_closest_centroids(X_large, centroids)

# 4. Remplacer les pixels par la couleur de leur centroid
# On utilise idx_large comme index pour piocher dans le tableau centroids
X_recovered = centroids[idx_large]

# 5. Remettre l'image sous forme de matrice (H x W x 3)
img_compressed = X_recovered.reshape(original_shape)

print("Compression terminée.")
```

```
Compression terminée.
```

[5]:
```python
fig, ax = plt.subplots(1, 2, figsize=(12, 6))

ax[0].imshow(img_large)
ax[0].set_title("Image Originale (Millions de couleurs)")
ax[0].axis('off')

ax[1].imshow(img_compressed)
ax[1].set_title(f"Image Compressée ({K} couleurs)")
ax[1].axis('off')

plt.show()
```